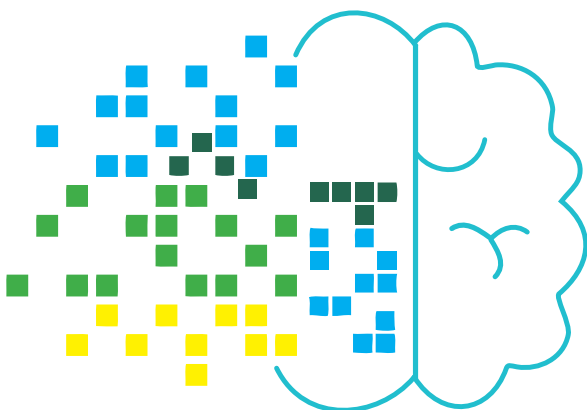


Memory Allocation Methods: An Overview

Dynamic memory management requires regular memory allocation and space clearance. This article lists a few methods that can be used for memory allocation.



Calling the malloc (memory allocation) or free function results in a reference to the allocated memory address, or it frees memory back for others to use each time it is called. Dynamic memory management necessitates regular memory allocation and space clearance.

What should be done with the space that has been freed up as well as with the space that has been used? How do we figure out which addresses/slots to utilise for the next malloc call? There are three approaches for managing memory that we can use to solve these questions, which we shall discuss in detail.

The table below illustrates the current state of one large memory block of 1024 bytes. We will see how each method uses it for the allocation of memory.

	0	250	550	700	900	950 - 1023
Address	0	250	550	700	900	950 - 1023
Allocation details	Block 1 allocated	Free 1	Block 2 allocated	Free 2	Block 3 allocated	Free 3
	Size = 250	Size = 300	Size = 150	Size = 200	Size = 50	Size = 73



Note: In this blog we will only consider the First Fit algorithm, though each of these methods has its own use cases. Generally speaking, preference is given to the First Fit.

First Fit

In the First Fit method, the free list is traversed sequentially to find the first free block that can hold the requested size.

Once the block is found, one of the two operations given below is performed:

- If the size is equal to the requested amount, it is removed from the free list.
- Else it is split into two parts. Here, the first portion remains on the list and the second is allocated.

The reason for allocating the second part is that the operation of updating the list may be avoided by just making a change in the size of the free node.

Let's now try to allocate 200 bytes and understand how the structure changes.

	0	250	350	550	700	900	950 - 1023
Address	0	250	350	550	700	900	950 - 1023
Allocation details	Block 1 allocated	Free 1	Block 2 allocated (new)	Block 3 allocated	Free 2	Block 4 allocated	Free 3
	Size = 250	Size = 100	Size = 200	Size = 150	Size = 200	Size = 50	Size = 73

As can be seen, the memory is allocated in the second section with the prior block pointing to the first part. If the first portion was used, the change would have had to be made in the list by pointing to the second section.

Now, let's examine the algorithm:

```
p = freeblock;
alloc = null; // pointer to store the allocated size n's address
q = null;

// find the free node which can allocate the given size n
while ( p != null && size(p) < n ) {
    q = p; // previous node
    p = next(p);
}

// if there is block large enough found
if ( p != null ) {
    s = size(p);
```